

APROXIMAČNÍ ALGORITMY

algoritmy běžící v polynomiálním čase, které najdou dobré řešení

Def. Optimalizační problém = $(I, \mathcal{F}, f, g)$

I = množina vstupů I (instancí)

$\mathcal{F}$ = funkce dávající přípustné řešení pro $\forall I \in I$

$\mathcal{F}(I)$ = množina přípustných řešení pro I

f = účelová funkce

$f: (I, s) \mapsto \mathbb{R} \quad (s \in \mathcal{F}(I))$

g = bit říkající, zda minimalizují či maximalizují f

Def. NP-optimalizační problém je optimalizační problém t.ž.

- délka přípustných řešení je nejvýš $\text{poly}(|I|)$
- jazyk dvojic (I, A) , $I \in I$, $A \in \mathcal{F}(I)$ je v P
 - ↳ polynomiální ověřitelnost validity řešení
- f je spočitatelná v polynomiálním čase

Def. Algoritmus A je R -aproximační, pokud:

- $A(I)$ běží v čase $\text{poly}(|I|)$ pro $\forall I \in I$
- min: $\forall I \in I: A(I) \leq R \cdot \text{OPT}(I)$
- max: $\forall I \in I: A(I) \geq \frac{1}{R} \cdot \text{OPT}(I)$

Problém obchodního cestujícího (TSP)

vstup: $V = \{1, \dots, n\}$

$d: V \times V \rightarrow \mathbb{R}_0^+$ (vzdálenosti)

výstup: $\tilde{G}$ permutace na V

cíl: minimalizace $\sum_{i=1}^{n-1} d(\tilde{G}(i), \tilde{G}(i+1)) + d(\tilde{G}(n), \tilde{G}(1))$

Metrický TSP

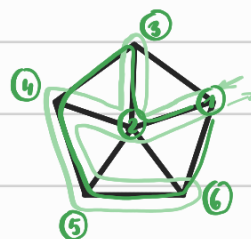
→ d je metrika:

- symetrie: $d(u, v) = d(v, u)$
- Δ nerovnost: $d(u, w) \leq d(u, v) + d(v, w)$
- axiom totožnosti: $d(u, v) = 0 \Leftrightarrow u = v$

Z (Zkrácení)

vstup: eulerovský multigraf (V, E)

1. Najdi eulerovský tah
2. C = cyklus $v_1, \dots, v_n$, kde vrcholy uspořádám podle prvního výskytu na eulerovském tahu



↳ C může obsahovat i hrany, které nejsou v E
ALE díky Δ nerovnosti to není problém

Lemma: $d(C) \leq d(E)$

Dk. z Δ nerovnosti

Kostrový algoritmus

vstup: metrika d na n vrcholech

1. T = minimální kostra v úplném grafu
2. $E = T \dot{\cup} T$ (disjunktů sjednocení)
↳ nahrazení $\forall$ hrany 2 kopiemi $\Rightarrow 2n-2$ hran

výstup: $Z(E)$

Lemma: $d(T) \leq \text{OPT}$

Dk.

bud' C nějaké přípustné řešení (např. OPT)

$D = C$ po odebrání 1 hrany

D je kostra: $d(T) \leq d(D) \leq d(C)$



□

Věta: Kostrový algoritmus je 2-aproximační

Dk.

$$d(C) \leq d(E) (=) 2 \cdot d(T) \leq 2 \cdot OPT$$

$$\hookrightarrow \text{def. } E = T \dot{\cup} T$$

□

Christofidesův algoritmus

vstup: metrika d na n vrcholech

1. Najdi minimální kostru T v úplném grafu

2. $W :=$ množina vrcholů lichého stupně v T ($|W|$ sudá)

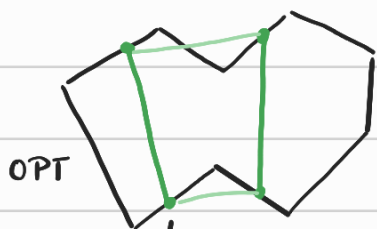
3. $M :=$ minimální perfektní párování v podgrafu indukovaném W

výstup: $Z(T \dot{\cup} M)$

$\hookrightarrow \nexists$ vrchol má sudý stupeň

Lemma: $d(M) \leq \frac{1}{2} OPT$

Dk.



$\rightarrow$ menší z párování jistě $\leq \frac{1}{2} OPT$

$\hookrightarrow$ vrcholy lichého stupně (cyklus na W) $\rightarrow$ jistě $\leq OPT$
(Δ nerovnost)

□

Věta: Christofidesův algoritmus je $\frac{3}{2}$ -aproximační

Dk.

$$ALG = d(Z(E)) \leq d(E) (=) d(T) + d(M) \leq OPT + \frac{1}{2} OPT = \frac{3}{2} OPT$$

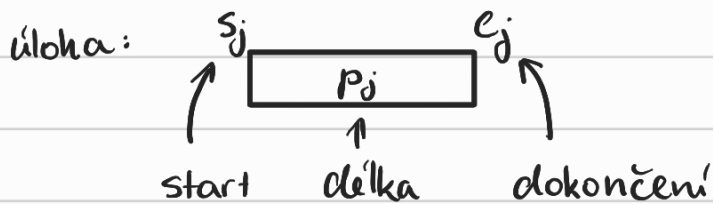
$$\downarrow \text{def. } E = T \dot{\cup} M$$

□

Rozvrhování

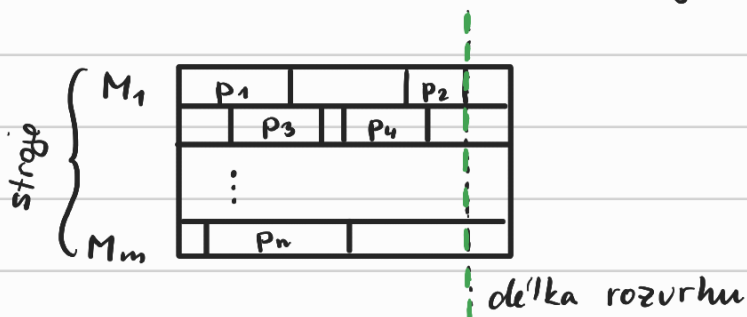
vstup: úlohy

výstup: rozvrh $\rightarrow$ rozvržení úloh na stroje



$\rightarrow$ start a dokončení přiřadí algoritmus

(lze přidat podmínky pro start a dokončení)



stroje mohou mít různé rychlosti, popř. mohou provádět pouze některé typy úloh

Rozvrhování na identických strojích

vstup: m (# strojů)

$p_1, \dots, p_n \geq 0$ (délky úloh)

výstup: rozklad $\{1, \dots, n\}$ na disjunktivní množiny $I_1, \dots, I_m$

I_i = úlohy na stroji

cíl: minimalizovat $\max_{i=1}^m \sum_{j \in I_i} p_j$

(úlohy skládám hned za sebe, žádné časové podmínky navíc)

Hladový algoritmus

1. $I_1, \dots, I_m \leftarrow \emptyset$

2. for $j = 1, \dots, n$:

3. rozvrhni j na nejmníh zatížený stroj

Věta: Hladový algoritmus je 2-aproximační

Dě.



nějaká úloha určující délku rozvrhu
 $T \leq \text{OPT}$ (z def. T)

$$p_j \leq \text{OPT} \Rightarrow T + p_j \leq 2\text{OPT}$$

T = doba, do které jsou všechny stroje používány

□

Věta: Hladový algoritmus na m strojích je $(2 - \frac{1}{m})$ -aproximační

Dě.

$$\left\{ \begin{array}{l} \frac{1}{m} \cdot \sum_{k=1}^n p_k \leq \text{OPT} \quad (\text{lépe než rovnoměrně úlohy rozvrhnout nelze}) \\ \sum_{k=1}^n p_k \geq m \cdot T + p_j \quad (\text{součet všech jistě menší než součet do } T + \\ \quad + \text{ úloha určující algoritmus}) \end{array} \right.$$



$$T + \frac{p_j}{m} \leq \text{OPT}$$

$$\underbrace{T + \frac{p_j}{m}}_{\text{ALG}} + \underbrace{\left(1 - \frac{1}{m}\right) p_j}_{(T + p_j)} \leq \underbrace{\text{OPT}}_{T + \frac{p_j}{m}} + \underbrace{\left(1 - \frac{1}{m}\right) \text{OPT}}_{(1 - \frac{1}{m}) p_j} = \left(2 - \frac{1}{m}\right) \text{OPT}$$

□

Lokální prohledávání

1. Najdi libovolný rozvrh (bez mezer)
2. Dokud lze přesunout:
3. pro stroj s maximálním C_j :
4. pokud $\exists$ stroj i s délkou rozvrhu $< S_j$:
5. přesuň úlohu j na stroj s min délkou rozvrhu
($S_j \leftarrow C_{\min}$)

☼ $C_{\min}$ neklesá

☼ $\forall$ úlohu přesuneme nejvýš 1

Důsledek: Algoritmus je polynomiální ($O(n)$ iterací)

Věta: Lokální prohledávání je 2-aproximační, pokud je m součástí vstupu
($(2 - \frac{1}{m})$ -aproximační pro pevné m)

Online algoritmy

- vstup přichází postupně
- řešení třeba konstruovat po krocích, pak už nejde měnit

rozvrhování: úloha přijde a musím ji hned rozvrhnout
např. hladový rozvrhovací algoritmus

Věta: Hladový algoritmus pro rozvrhování na 2 nebo 3 strojích je optimální online algoritmus

LPT (largest processing time first)

1. uspořádáme úlohy od nejdelší po nejkratší
($p_1 \geq p_2 \geq \dots \geq p_n$)
2. použijeme hladový algoritmus

Věta: LPT je $\frac{4}{3}$ -aproximační (resp. $\frac{4}{3} - \frac{1}{3m}$ pro pevné m)

Dě.

BÚNO: p_n určuje délku rozvrhu

$$\textcircled{1} \quad p_n \leq \frac{1}{3} \text{OPT} \quad \text{tzn. } \forall i: p_i \leq \frac{1}{3} \text{OPT} \quad (\text{podobné jako u hladového a.})$$

$$\text{ALG} = T + p_n, \quad T + \frac{p_n}{m} \leq \text{OPT}$$

↓

$$T + p_n \leq \text{OPT} + \left(1 - \frac{1}{m}\right) \frac{1}{3} \text{OPT} = \left(\frac{4}{3} - \frac{1}{3m}\right) \text{OPT}$$

$$\textcircled{2} \quad p_n > \frac{1}{3} \text{OPT} \quad (\text{tzn. } \exists i: p_i > \frac{1}{3} \text{OPT})$$

→ pak LPT vygeneruje optimální rozvrh

$$\bullet \quad \forall i: p_i > \frac{1}{3} \text{OPT} \Rightarrow \forall \text{stroj}: \leq 2 \text{ úlohy}$$

p_1	
p_2	
$\vdots$	
$\vdots$	
p_{m-1}	p_{m+2}
p_m	p_{m+1}

↪ $n \geq m+1$, alespoň na 1 stroji 2 úlohy

$$p_m + p_{m+1} \leq \text{OPT}$$

$$p_{m-1} + p_{m+2} \leq \text{OPT}$$

: analogicky dále

□

Bin packing

vstup: $a_1, \dots, a_n \geq 0$ ($\in [0,1]$)

výstup: rozklad $\{1, \dots, n\}$ na disjunktivní podmnožiny $I_1, \dots, I_m$

$$\text{t.j. } \forall i \sum_{j \in I_i} a_j \leq 1$$

cíl: minimalizovat m

ANYFIT (rodina algoritmů)

1. $m = 0$

2. $j = 1, \dots, n$:

 dáme j do nějakého I_i , kam se vejde

 → pokud takové I_i neexistuje:

$$m = m + 1$$

$$I_m = \{j\}$$

FIRSTFIT

→ dej a_j do 1. koše, kam se vejde

BESTFIT

→ dej a_j do nejplnějšího koše

Věta: FIRSTFIT ; BESTFIT jsou 1,7-aproximační algoritmy

Věta: Je NP-těžké najít R -aproximační algoritmus pro $R \leq \frac{3}{2}$

Dle je NP-těžké rozhodnout, zda $OPT = 2$

(dělení na 2 množiny se stejným součtem → NP těžké)

Věta: $\forall \epsilon \exists \text{ALG } \forall I \text{ ALG}(I) \leq (1+\epsilon)OPT(I) + 1$

Věta: $\forall$ ANYFIT algoritmus má aproximační poměr ≤ 2

(nelze říci 2-aproximační, protože bychom museli předpokládat polynomiální čas)

Věta: Hladový algoritmus s $c=1$ je $O(\sqrt{m})$ -aproximační

Dě.

můžeme předpokládat $OPT \geq 1 \Rightarrow |I| = ALG \geq 1$ ($\exists$ cesta)

bud' I^* , $\{P_i^* | i \in I^*\}$ optimální řešení

- dlouhé cesty: $|P_i^*| > \sqrt{m}$

$\rightarrow$ je jich $\leq \sqrt{m}$ (jinak $> m$ hran)

- krátké cesty

$i \in I \checkmark$

$i \notin I \rightarrow P_i^*$ má společnou hranu s nějakou cestou P_j

t.č. $|P_j| \leq \sqrt{m}$ (jinak by P_i^* vybral dřív)

$\hookrightarrow \# \text{ krátkých cest } \leq |I| \cdot (\sqrt{m} + 1)$

$$|I^*| \leq \sqrt{m} + |I|(\sqrt{m} + 1) \leq O(\sqrt{m})|I|$$

□

$\hookrightarrow$ kvůli $\lceil \sqrt{m} \rceil$

hladový algoritmus (s kapacitami)

$\rightarrow$ po t použití hrany ji prodloužím faktorem β

pro $c=1$: $\beta = \sqrt{m}$

1. $\beta := \lceil m^{\frac{1}{c+1}} \rceil$

2. $I := \emptyset$, $d(e) := 1 \quad \forall e \in E$

3. najdi nejkratší P_i vzhledem ke vzdálenosti d

4. pokud $\exists$ a $d(P_i) < \beta^c$:

5. $I := I \cup \{i\}$

6. $\forall e \in P_i: d(e) = d(e) \cdot \beta$

7. jinak:

vystup I , $\{P_i | i \in I\}$

☞ algoritmus nepoužije e s $d(e) \geq m$

→ je polynomiální

→ výsledné řešení je přípustné

(po c použitích je $d(e) = \beta^c \approx m^{\frac{c}{c+1}} \approx \frac{m}{\beta}$)

Věta: Hladový algoritmus s kapacitami je $O(m^{\frac{1}{c+1}})$ -aproximační

Dě.

$$\text{BÚNO } \text{OPT} \geq 1 \rightarrow |I| = \text{ALG} \geq 1$$

$$P_i^*, i \in I \quad \checkmark$$

$$P_i^*, i \notin I: \Rightarrow d(P_i^*) \geq \beta^c \text{ (jinak bych ji použila)}$$

Spodní odhad:

$$\sum_{P_i^*, i \notin I} d(P_i^*) \geq \beta^c (|\text{OPT}| - |I|)$$

↳ každá cesta délky $\geq \beta^c$ a je jich $\geq (|\text{OPT}| - |I|)$



?

$$d(E) \geq \beta^c (|\text{OPT}| - |I|) \cdot \frac{1}{c}$$

↳ každou hranu můžeme použít c -krát

horní odhad:

$$\text{na začátku: } d(E) = m$$

$$\text{po výběru } P_i: d(P_i) \leq \beta^c \cdot \beta = \beta^{c+1}$$

$$\text{na konci: } d(E) \leq m + |I| \beta^{c+1} \leq (|I| + 1) \beta^{c+1}$$

?

$$\rightarrow \beta^c (|\text{OPT}| - |I|) \leq c \cdot d(E) \leq c (|I| + 1) \beta^{c+1} \quad / : \beta^c$$

$$|\text{OPT}| - |I| \leq \beta c (|I| + 1)$$

$$|\text{OPT}| \leq O(\beta) \cdot |I|$$

□

• s rostoucí kapacitou se algoritmus zlepšuje

Vrcholové pokrytí

vstup: $G = (V, E)$

ceny $c_v \geq 0 \quad \forall v \in V$

výstup: $W \subseteq V$ t.ž. $\forall e: e \cap W \neq \emptyset$

cíl: minimalizuj $c(W) = \sum_{v \in W} c_v$

Celčíselný program

proměnné: $x_v \in \{0, 1\}$

podmínky: $x_u + x_v \geq 1 \quad (uv = e \in E)$

účelová fce: $\min \sum_v c_v x_v$

LP relaxace $\rightarrow x_v \geq 0$

VC-LP

1. Najdi x^* optimum LP

2. $W = \{v \in V \mid x_v^* \geq \frac{1}{2}\}$

$\hookrightarrow$ přípustné: pro splnění podmínek vždy ≥ 1 z $(x_u, x_v) \geq \frac{1}{2}$

Věta: VC-LP je 2-aproximační pro VP

Dle. lze implementovat v P

W je přípustné řešení

$$\sum_{v \in W} c_v \leq \sum_{v \in W} 2 \cdot c_v x_v^* = 2 \cdot \sum_{v \in W} c_v x_v^* = 2 \cdot \text{OPT}_{LP}$$

$\hookrightarrow$ proměnné $x \geq \frac{1}{2}$ zaokrouhluji na 1 $\rightarrow$ max dvojnásobek $\square$

asi neexistuje lepší než 2-aproximační algoritmus

W je VC $\Leftrightarrow V \setminus W$ je nezávislá

nezávislá množina nemá $n^{1-\epsilon}$ -aproximační algoritmus

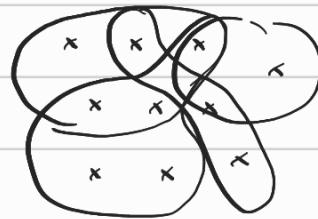
Množinové pokrytí (set cover)

vstup: $S_1, \dots, S_m \subseteq \{1, \dots, n\}$

$$c_1, \dots, c_m \geq 0$$

výstup: $I \subseteq \{1, \dots, m\}$ t.č. $\bigcup_{i \in I} S_i = \{1, \dots, n\}$

cíl: minimalizovat $\sum_{i \in I} c_i$



parametry: $q = \max_{j=1}^m |S_j| \leq n \rightarrow$ velikost největší množiny
 $f = \max_{e=1}^n |\{j \mid e \in S_j\}| \rightarrow$ max # výskytů 1 prvku

VC je SETCOVER $\leq f \leq 2$

(S_j odpovídá hranám z vrcholu j

$\rightarrow \forall$ hrana se objeví ve 2 množinách)

lineární program

proměnné: $x_1, \dots, x_m \geq 0$

podmínky: $\forall e=1, \dots, n : \sum_{j: e \in S_j} x_j \geq 1$

účelová fce: $\min \sum_{i=1}^m x_i c_i$

SC-LP

1. Najdu x^* optimum lineárního programu

2. $I = \{j \mid x_j^* \geq \frac{1}{f}\}$

Věta: SCLP je f -aproximační.

Dk. e je pokrytý $\checkmark \rightarrow$ z podmínek

$$ALG = \sum_{i \in I} c_i \leq f \cdot \sum_{i \in I} c_i x_i^* \leq f \cdot \sum_{i=1}^m c_i x_i^* = f \cdot OPT_{LP}$$

$\hookrightarrow$ proměnné zvýším nejvýše f -krát

□

Def. Dualní program k lineárnímu programu

$$x_1, \dots, x_m \geq 0$$

$$\forall e \in \{1, \dots, n\} : \sum_{j \in S_j} x_j \geq 1$$

$$\min \sum_{i=1}^m x_i c_i$$

$$y_1, \dots, y_n \geq 0$$

$$\forall j \in \{1, \dots, m\} : \sum_{e \in S_j} y_e \leq c_j$$

$$\max \sum_{e \in S_j} y_e$$

Def. Podmínky komplementarity

x^*, y^* jsou optimální řešení LP a jeho dualu:

$$\bullet \forall j : x_j^* = 0 \vee \sum_{e \in S_j} y_e^* = c_j$$

$$\bullet \forall e : y_e^* = 0 \vee \sum_{j \in S_j} x_j^* = 1$$

Věta: (slabá věta o dualitě)

Pro LP a jeho DP:

maximalizující účelová fce $\leq$ minimalizující účelová fce
(v optimu nastává rovnost)

Primárně-dualní algoritmus (PDSC)

$$1. y_1, \dots, y_n := 0$$

$$I := \emptyset, E := \emptyset$$

2. dokud $\exists e \notin E$: „zvýšíme y_e co nejvíce“

$$\delta = \min_{j \in S_j} (c_j - \sum_{e \in S_j} y_e)$$

$$y_e = y_e + \delta$$

$$\forall j : e \in S_j \wedge \sum_{e \in S_j} y_e = c_j : I := I \cup \{j\} \wedge E = E \cup S_j$$

Věta: Algoritmus je f -aproximační

$$\text{Dle. } \underset{\substack{\uparrow \\ \text{def}}}{\text{ALG}} = \sum_{j \in I} c_j = \sum_{j \in I} \sum_{e \in S_j} y_e \leq f \cdot \sum_{e=1}^n y_e \leq f \cdot \text{OPT}$$

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
 def def ? $\hookrightarrow$ dualní řešení

Lemma: $(\forall j \in \{1, \dots, n\}) \sum_{i \in S_j} q_i \leq c_j$

co to kurva je a
kde se to tu vzab?

Dle vezmu pevné j , $S_j = \{x_1, \dots, x_k\}$
očíslovu tak, že pokryjí v pořadí $x_k, x_{k-1}, \dots, x_1$

Hladový algoritmus

$q :=$ vektor indexovaný prvky

$\rightarrow$ odpovídá ceně za pokrytí daného prvku

1. $I := \emptyset$, $E := \emptyset$

2. Dokud $\exists e \notin E$:

$$p_j = \frac{c_j}{|S_j \setminus E|} \quad \text{pro } j: S_j \not\subseteq E$$

j_0 t.ž. p_{j_0} je minimální

$$q_e := p_{j_0} \quad \forall e \in S_{j_0} \setminus E$$

$$I = I \cup \{j_0\}$$

$$E = E \cup S_{j_0}$$

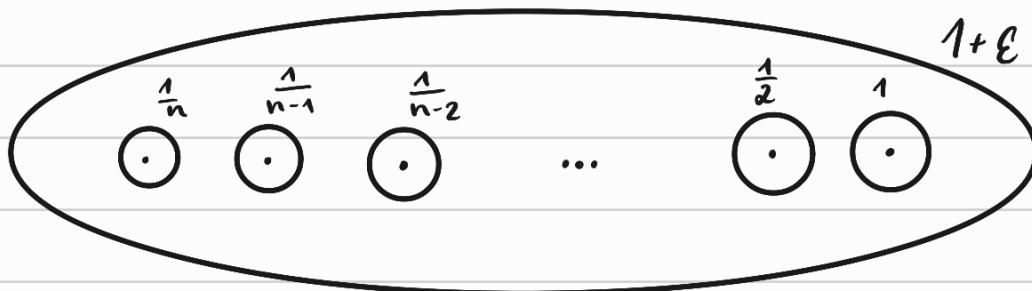
3. výstup: I

Věta: Hladový algoritmus je H_g -aproximační

$$H_g = 1 + \frac{1}{2} + \dots + \frac{1}{g} \approx \ln g \leq \ln n \quad (\text{harmonické číslo})$$

$$\star \text{ ALG} = \sum_{e=1}^n q_e$$

$\hookrightarrow$ cena p_i při přidávání rozdělím do q_e



Lemma: $\bar{q} = \frac{1}{H_j} \cdot q$ je přípustné řešení duálního LP
Dk.

chceme: $\sum_{e \in S_j} \bar{q}_e \leq c_j$ (podmínka duálu)

necht' $S_j = \{e_1, \dots, e_k\}$

$e_k = 1.$ pokrytý, ..., $e_1 =$ poslední pokrytý

$$\odot q_{e_i} \leq \frac{c_j}{i}$$

- v i -tém kroku nejsou pokryté prvky $1, \dots, i$
- z definice vybíráme nejlevnější možnou množinu

$$\sum_{e \in S_j} q_e = \sum_{i=1}^k q_{e_i} \leq \frac{c_j}{1} + \frac{c_j}{2} + \dots + \frac{c_j}{k} = c_j \cdot H_k$$

↓

$$\sum_{e \in S_j} \bar{q}_e = \frac{1}{H_j} \cdot \sum_{e \in S_j} q_e \leq c_j \quad \checkmark$$

□